VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI
School of Computer Science and Engineering

SMART RIDESHARE RECOMMENDATION AND ANALYTICS SYSTEM

Using Java, Python, MySQL and Machine Learning

A Project Report

submitted in partial fulfilment of the requirements for the degrees of
Master of Technology

Submitted by

Godwin Durai Raja A (26MAI1021)
M.Tech CSE (Artificial Intelligence and Machine Learning)

Raghul T (26MCB1023)
M.Tech Big Data Analytics

October 2026

ABSTRACT

Daily commuters often travel along similar routes at similar times while available vehicle seats remain unused. Coordinating these journeys through informal messages makes it difficult to discover suitable rides, compare departure times, manage capacity and retain a reliable record of participation. This project develops Smart RideShare, a local web application that combines ride offering, passenger requests, automatic recommendations, map-based route calculation, booking management and analytical reporting. The interface uses HTML, CSS and JavaScript, while Java Servlets and JSP run on Apache Tomcat to manage authentication, authorisation and transactional operations. MySQL stores users, rides, requests, bookings, feedback, conversations and simulated payments. A separate Python service uses scikit-learn for recommendation scoring and exploratory demand forecasting. Candidate rides are first filtered by ownership, availability, departure time and endpoint proximity. Eligible rides are ranked using five features: pickup proximity, destination proximity, timing compatibility, conversation preference and driver rating. A similarity-based baseline supports cold start; a Random Forest model contributes to ranking when sufficient valid feedback is available. Leaflet displays map interaction, and OSRM supplies road geometry and distance. The server calculates a per-seat contribution at INR 5 per kilometre and rejects stale client-side values. Saved requests refresh automatically while the interface is active, but passengers explicitly confirm bookings. Participant-only chat supports pickup coordination, and a clearly labelled local payment simulator demonstrates successful, failed, cancelled and refunded payment states without external payment APIs or money movement. The completed implementation was checked through 59 automated assertions across Java security tests, Python matching tests, application integration, map and recommendation workflows, chat and payments. Browser checks also covered mobile layout and JavaScript errors. These results demonstrate functional behaviour in the local environment; they do not establish predictive accuracy on a real commuter population. Demonstration data includes synthetic records, and carbon, cost and demand outputs are estimates. The project provides an integrated academic prototype for shared commuting and a foundation for future evaluation using consented operational data.

1. INTRODUCTION

1.1 Background and Motivation

A shared commute begins with a simple opportunity: one person has an available seat and another person needs to travel in a compatible direction. In a college or corporate community, recurring arrival times and common destinations can make such opportunities practical. However, a useful coordination tool must consider more than the names of two locations. It must account for actual coordinates, departure time, remaining capacity, user preferences and changes made after a ride is published. A list of offers alone cannot reliably handle all of these concerns.

Smart RideShare brings these steps into one application. A driver publishes a journey, a passenger saves a request, and the recommendation service identifies compatible offers. A map explains the selected route and its distance, while transactional booking prevents over-allocation of seats. The application also keeps participants informed through chat and stores completed journeys for analysis. This combination makes the project suitable for demonstrating how web engineering, relational databases and machine learning can cooperate in a single workflow.

1.2 Problem Statement

The problem addressed is to build a working, understandable ride coordination system that automatically ranks feasible ride offers for a passenger while maintaining consistent booking records. Location changes must produce corresponding distance and price changes. A recommendation must not include a full ride, the passenger's own offer, an expired departure or a route outside the permitted matching window. The system must also distinguish between an absence of suitable offers and a technical failure, protect account-specific actions and present analytical estimates without misrepresenting them as measured real-world outcomes.

1.3 Objectives

- **Integrated ride workflow:** implement registration, login, ride offering, passenger requests, booking, cancellation, completion and feedback.
- **Automatic recommendations:** combine feasibility checks, explainable similarity scoring and feedback-based learning to rank available rides.
- **Location-aware coordination:** display road routes, calculate distance-based fares and provide participant-only communication.

- **Verification and reporting:** validate access controls, booking consistency, automatic updates and demo payment states; present usage, cost and environmental estimates with explicit assumptions.

1.4 Scope

The implemented scope is a locally hosted academic prototype with employee and administrator roles. It includes a Ride Module, an Analytics Module, an Admin Module, a Python recommendation service, a road map, group chat and simulated payment checkout. The Java project uses a Maven layout that can be imported into Eclipse. Tomcat, the Python service and a dedicated MySQL instance run on the same development computer.

Real payment collection, driver payouts, vehicle tracking and traffic-aware arrival prediction are outside the implemented scope. Payment outcomes are intentionally simulated, with no merchant account or external payment gateway. Road routing and map tiles still depend on internet access. The recommendation model is evaluated for functional behaviour, not for predictive accuracy on an independently labelled real-world dataset. Its score is a ranking signal rather than a probability of passenger satisfaction.

2. LITERATURE REVIEW

Dynamic ride-sharing combines a transportation problem with an information-system problem. A rider and driver must be compatible in space and time, but the system also needs to maintain availability as users create, change or cancel plans. Agatz et al. [1] review optimisation approaches for dynamic ride-sharing and discuss the challenges created by short-notice matching. Their work establishes the relevance of explicit constraints and timely decisions. For the present project, this motivates a two-stage approach: first remove infeasible candidates, then rank the remaining offers. Smart RideShare does not reproduce a fleet-wide optimisation algorithm; it applies this separation to a smaller community application.

Santi et al. [2] introduce shareability networks to study when trips can be combined under limits on passenger inconvenience. Their framework connects feasible trip combinations to potential collective benefits. The relevant lesson for this implementation is that sharing opportunities depend on the permitted spatial and temporal deviation, rather than on a location label alone. Smart RideShare therefore stores coordinates and checks both endpoint gaps and a departure tolerance. Its environmental dashboard is a simplified estimate based on completed shared travel, not a reproduction of the network study or a measured city-wide reduction.

Alonso-Mora et al. [3] describe dynamic trip-vehicle assignment for on-demand high-capacity ride-sharing. Their work addresses combinations of requests and vehicles under service constraints. This is broader than the current prototype, where a driver publishes a fixed journey and passengers select individual available seats. The comparison identifies a deliberate boundary: Smart RideShare recommends compatible existing offers but does not optimise the sequence of intermediate pickups, dispatch a managed fleet or guarantee a globally optimal assignment. Such capabilities would require a richer routing and scheduling model.

Breiman's Random Forest method [4] combines multiple decision trees. This provides a useful basis for modelling nonlinear relationships without committing to a single tree. The project applies a Random Forest regressor to five bounded matching features, using completed-ride feedback as the response. The use of a learned model does not by itself establish good recommendations: the quality, representativeness and timing of the feedback remain important. A transparent baseline is retained so the application can operate before enough valid samples have been collected.

The scikit-learn project described by Pedregosa et al. [5] supplies reusable machine-learning components for Python. In Smart RideShare it supports both cosine similarity and Random Forest regression. Keeping these calculations in a separate service creates a clear division between recommendation logic and the Java booking workflow. It also makes it possible to test eligibility and scoring behaviour independently of the browser. The current tests demonstrate implementation behaviour; an additional empirical study would be necessary to compare ranking methods on unseen commuters.

Mapping introduces another distinction: a straight-line distance between two coordinates is useful for comparing pickup proximity, but it is not the same as the length of a driving route. The OSRM route-service documentation [6] describes route geometry, distance and duration outputs. Smart RideShare uses this service for the displayed road route and fare calculation. Leaflet [7] provides the browser-side map layers, markers and interaction. The implementation keeps these responsibilities separate: users choose locations in the browser, while Java obtains and validates the authoritative route quote.

Reliable application state is equally important. MySQL's InnoDB documentation [8] explains transaction locking, which supports the project's final-seat booking safeguard. A booking transaction locks the relevant ride before changing its available capacity. Apache Tomcat [9] provides the Servlet and JSP runtime used for the application. These platform capabilities enable the project to combine persistent records with authenticated web requests, although they do not replace the need for application-level ownership checks.

The resulting design combines ideas from constrained matching, feedback-driven ranking, interactive mapping and transactional application development. Its contribution is the integration of these mechanisms into a complete teaching demonstration: a request can become a recommendation, a recommendation can become a booking, and a completed booking can provide feedback and analytical records. Chat and simulated payments extend the coordination workflow. The project makes no claim of a new ride-sharing optimisation theorem or a validated forecasting model; its current evidence is an executable implementation with repeatable functional checks.

3. METHODOLOGY

The implementation follows five stages: (1) capture authenticated user and journey data, (2) validate locations and availability, (3) filter and rank candidate rides, (4) confirm and manage bookings, and (5) record feedback and derive analytical estimates. The following sections describe the data and configuration used in the local implementation.

3.1 Dataset

The application uses structured MySQL records rather than an image-classification dataset. Seeded demonstration history is synthetic; subsequent user records and isolated test fixtures are separate sources. The test scripts remove their fixture accounts and journeys after execution. Table 1 groups the eleven persisted application tables by responsibility.

Data group	Tables	Purpose
Identity and governance	3	Users, settings, audit log
Ride lifecycle and learning	5	Rides, requests, bookings, matches, feedback
Coordination	2	Messages and read cursors
Demo payments	1	Attempts, amounts and outcomes

Table 1. Persisted data groups in the completed application (11 tables).

No random train/validation/test split is claimed for the operational records. Functional validation uses controlled cases with known expected behaviour. Feedback training requires at least ten valid examples; forecasting requires at least fourteen observed calendar days. Table 2 records the principal evaluation boundaries.

Evaluation component	Input	Reported output
Functional checks	Isolated controlled fixtures	Pass/fail assertions
Feedback ranking	Five features and ride rating	Suitability score
Demand forecasting	Daily/hourly request history	Exploratory 7-day estimate

Table 2. Evaluation inputs and outputs; no held-out predictive accuracy is claimed.

3.2 Data Validation and Feature Engineering

Java validates required fields, coordinate ranges, future departures and seat limits before persisting a journey. The server calculates the road distance and fare rather than trusting values submitted by the browser. Invalid or identical endpoints clear the displayed quote.

For matching, the Python service computes Haversine distances between the requested and offered pickup points and between their destinations. It also calculates the absolute departure-time difference. Five normalised features represent pickup proximity, destination proximity, timing, conversation compatibility and driver rating. Distance and time constraints are applied before ranking; a high rating cannot override an infeasible route or a full vehicle.

3.3 System Architecture and Recommendation Model

The browser communicates with authenticated Java endpoints. Java uses JDBC for MySQL operations, calls the private Python service for recommendations and forecasts, and requests road quotes from OSRM. The Python endpoint requires a service key and is bound to the local machine. Eclipse provides the development environment; Maven packages the application as a WAR for Tomcat. Figure 1 summarises the implemented data flow.

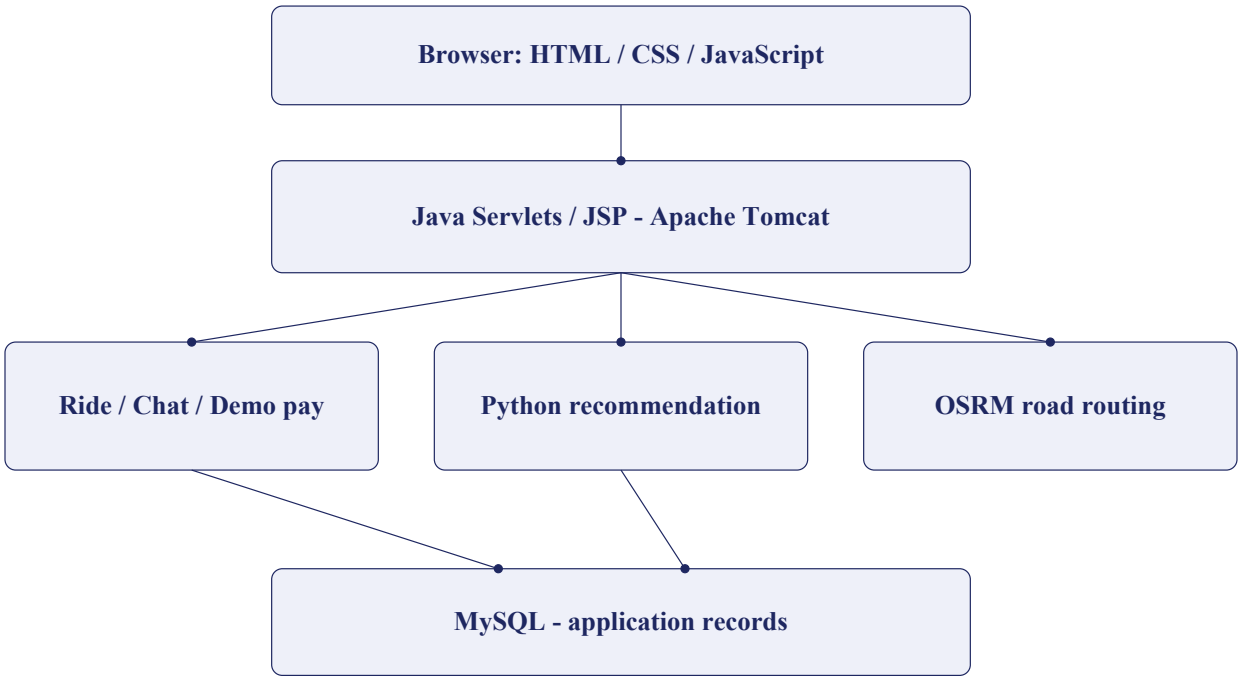


Figure 1. Implemented Smart RideShare architecture and service boundaries.

3.4 Matching and Training Strategy

Cold-start ranking blends weighted feature magnitude with cosine similarity. Once sufficient valid feedback exists, the feedback-trained regressor contributes to the final suitability score. Feasibility checks remain mandatory in both modes.

- **Candidate filtering:** exclude duplicate offers, the requester's own rides, past departures, closed rides and rides without seats. Pickup and destination gaps must each be within the configured threshold; the departure difference must fit the passenger's tolerance.
- **Baseline score:** use weights of 0.30 for pickup proximity, 0.30 for destination proximity, 0.25 for timing, 0.10 for conversation compatibility and 0.05 for driver rating. Blend 80% weighted magnitude with 20% cosine similarity.
- **Feedback learning:** map ratings from 1-5 to 0-1 and fit a Random Forest regressor with 80 trees, maximum depth 5, minimum leaf size 2 and random seed 42. Blend 60% baseline and 40% model prediction; clamp and display the result on a 0-100 scale.

A versioned model file is published after training. Matching results retain the feature values and model version to support explanation. The application checks saved upcoming requests every five seconds while the page is visible. Recommendations do not reserve capacity: a separate booking transaction checks availability again before allocating a seat. This avoids treating a previously displayed recommendation as a guarantee that a seat remains available.

The demand model uses weekday and hour as inputs and includes zero-demand hours within a maximum ninety-day history window. It returns seven daily estimates when sufficient history exists. This forecast is available in the administrator's analytics view and is labelled exploratory. No held-out forecast error is reported in this project.

3.5 Evaluation Metrics

The primary evaluation metric is the number of automated assertions passed in each functional suite. Assertions cover authorisation, validation, lifecycle transitions, automatic refresh, data persistence and concurrency. Java tests validate security and input helpers; Python tests validate recommendation behaviour; full-stack scripts use HTTP requests and real browser contexts. Controlled inputs make expected outcomes explicit, including the rejection of unsuitable rides and exactly one winner for a final seat.

Distance and fare verification compares the UI with the server quote and checks that changing locations changes the values. Chat checks cover participant membership, delivery, history and escaped message rendering. Payment checks cover server-derived amounts, retry behaviour, receipts and simulated refunds. Predictive measures such as precision, recall, F1, ranking relevance and forecast error would require a separate labelled evaluation dataset; none is invented from these software checks.

4. RESULTS AND DISCUSSION

4.1 Functional Validation and Model Status

The final local application passed 59 automated checks: 3 Java checks, 6 Python checks, 17 application integration checks, 12 map and automatic-matching checks, 11 ride-chat checks and 10 simulated-payment checks. The Java build completed successfully and produced a deployable WAR. Browser smoke checks additionally rendered the main modules without JavaScript errors and verified a narrow mobile viewport. Figure 2 summarises the automated suite counts.

The integration results verified registration and password login, protected administrator actions, CSRF enforcement, input validation and ride lifecycle operations. The concurrent booking test produced exactly one successful reservation for the final seat. Cancelling a booking released one seat, while cancelling a driver's ride propagated the state to its passenger bookings. Feedback was accepted only for an eligible completed booking and duplicate feedback was rejected.

The map and matching workflow used live local MySQL offers. A newly published compatible offer appeared automatically for a saved passenger request, and the passenger could book it. Server-side fare recomputation rejected stale or tampered submitted values. One test run encountered an external routing timeout; a subsequent complete map run passed all twelve checks. This observation documents the dependence on routing-service availability rather than implying uninterrupted network service.

The model service was available and feedback training executed during integration testing. This confirms that the learning pipeline is connected, not that the model predicts passenger satisfaction accurately. The demo history includes synthetic feedback and is unsuitable for a claim of real-world generalisation. All reported counts in this section are software verification results, not classification accuracy or a commuter trial.

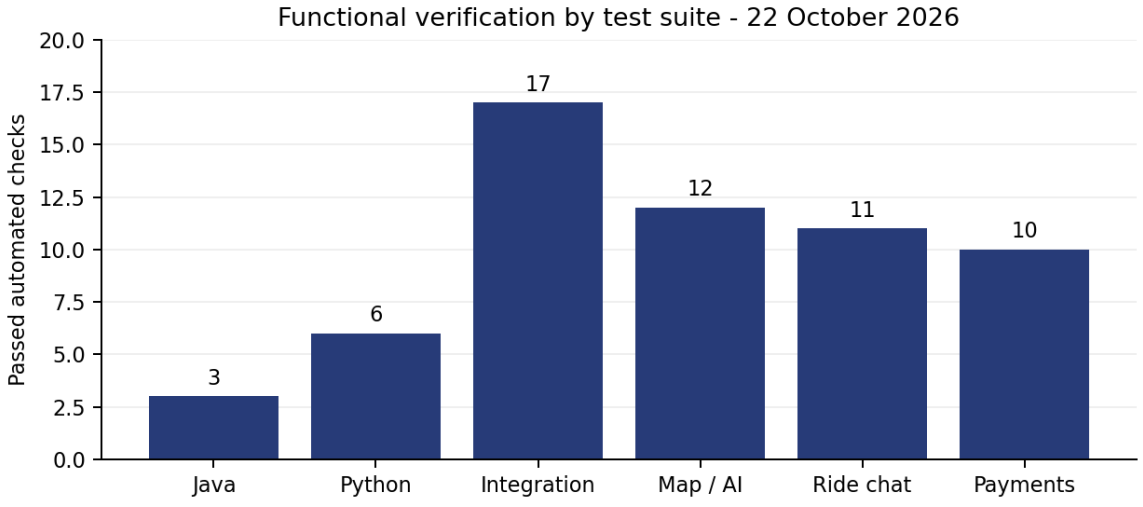


Figure 2. Passed automated checks by suite (59 total); counts are not AI accuracy.

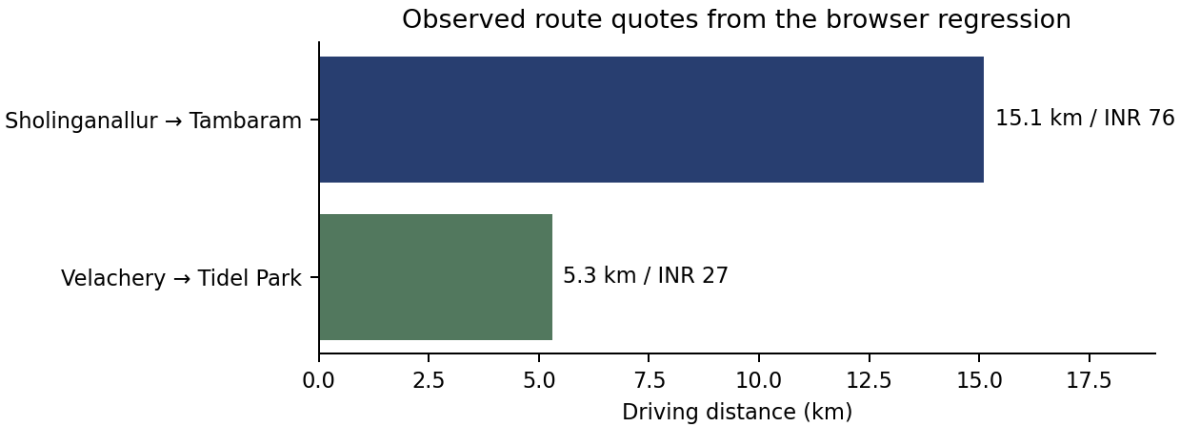


Figure 3. Road-distance and fare quotes observed in the map regression test.

4.2 Integration Test Performance

Table 3 records the final passing result for each suite. The route examples in Figure 3 changed from 5.3 km and INR 27 per seat for Velachery to Tidel Park to 15.1 km and INR 76 for Sholinganallur to Tambaram. The quote uses the selected coordinates and the returned road route. These values are observations from the tested configuration; future route data may produce different results.

Suite	Checks passed	Result
Java security / input	3 / 3	Pass
Python matching / forecast	6 / 6	Pass
Full-stack integration	17 / 17	Pass
Map and automatic matching	12 / 12	Pass
Ride chat	11 / 11	Pass
Simulated payments	10 / 10	Pass
Total	59 / 59	Pass

Table 3. Local automated verification results, 22 October 2026.

4.3 Matching Decision Analysis

A zero-match result is not automatically a model failure. A passenger may have selected a time for which no offer exists, may be viewing their own offers or may have coordinates outside the configured endpoint threshold. The implementation reports diagnostic reasons and continues checking saved requests. Table 4 and Figure 4 illustrate controlled eligibility cases exercised by the Python test suite.

The matching service checks feasibility before computing a score. In the controlled near-match case, the compatible offer was included once even when supplied twice. Separate cases verified exclusion for excessive pickup distance, self-owned offers, full rides and departure times outside the tolerance. These observations test defined rules; they are not a confusion matrix of learned predictions against human-labelled commuter preferences.

Controlled case	Expected decision	Observed
Compatible nearby offer	Include	Include
Pickup outside threshold	Exclude	Exclude
Requester is driver	Exclude	Exclude
No remaining seats	Exclude	Exclude
Departure outside tolerance	Exclude	Exclude

Table 4. Controlled eligibility outcomes; these are assertions, not predictive classes.

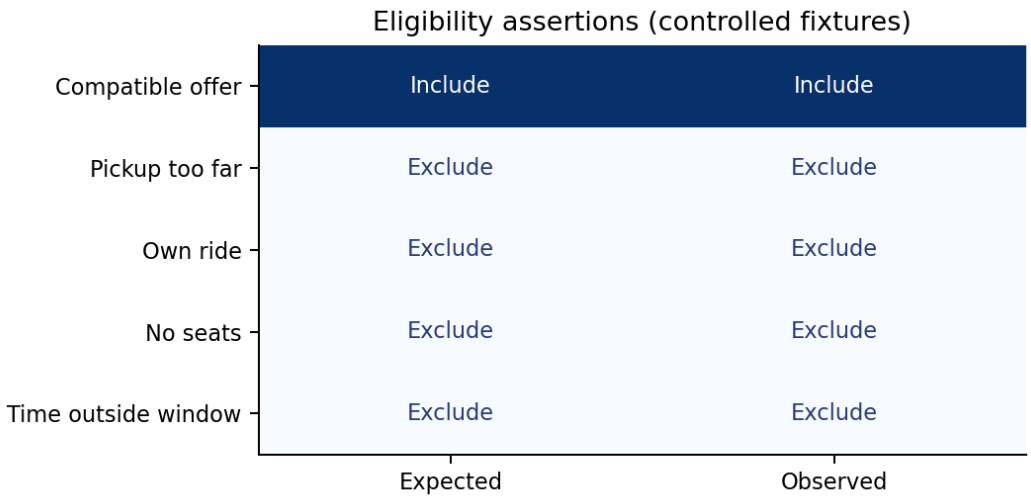


Figure 4. Agreement between expected and observed eligibility decisions.

4.4 Qualitative Analysis

The interface supports a continuous demonstration from login to completion. The Ride Module displays map controls and recommendations, while My journeys groups offers, bookings and saved requests. Users can coordinate pickup through a ride-specific conversation. Figure 5 shows actual application captures, including the new payment simulator. Screenshots represent the local demonstration rather than a production deployment.

Chat delivery was verified between two independent browser contexts, with drafts retained during updates. Nonparticipants could not read or send messages. Simulated checkout recorded success, failure and cancellation without collecting bank details. A successful demo payment produced a downloadable receipt; cancelling the booking or ride recorded a simulated refund. No external payment gateway was contacted.

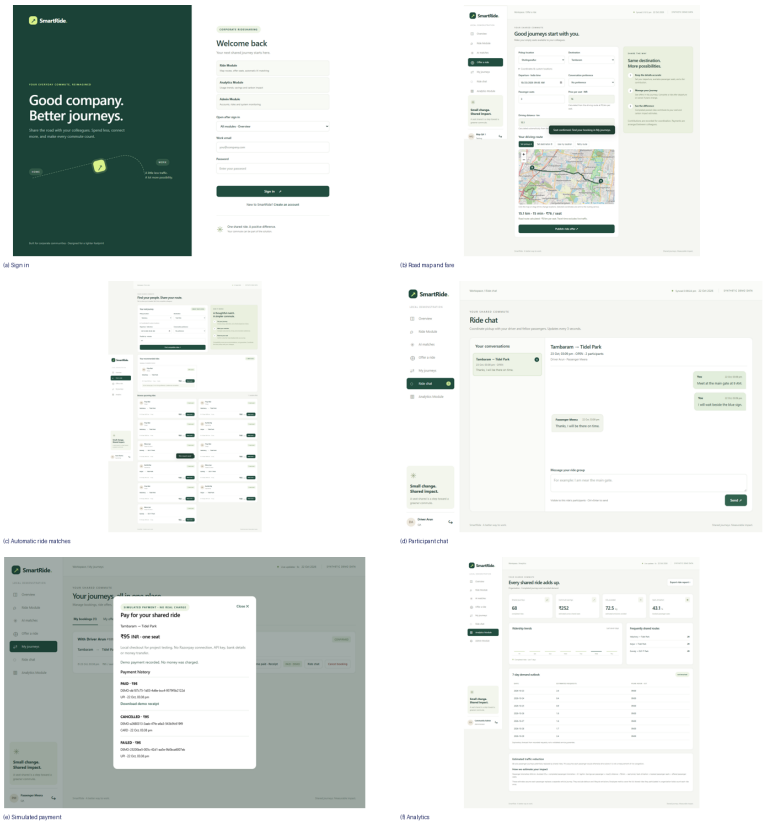


Figure 5. Actual application views: login, routing, matching, chat, demo payment and analytics.

4.5 Discussion

The strongest result is the consistency of the complete workflow. Recommendations use current offers, bookings protect seat capacity, and subsequent chat and payment actions are tied to the authenticated passenger or ride membership. Repeated payment completion does not create a second successful charge record, and repeated chat submission with the same message key does not duplicate the message. These checks address practical failure cases that would be missed by reviewing screenshots alone.

The main limitations are data quality and external routing availability. Synthetic histories demonstrate the analytics but cannot establish real demand or savings. Endpoint proximity does not calculate the exact detour a driver would make for an intermediate pickup. Polling provides near-real-time interface updates rather than continuous vehicle tracking. The payment feature is a teaching simulator, not a Razorpay integration. These boundaries guide the next evaluation and deployment steps.

5. CONCLUSION AND FUTURE WORK

This project implements Smart RideShare as an integrated local web application for coordinating shared commutes. The completed system provides registration and login, map-based ride offering, passenger requests, automatic recommendations, transactional seat booking, cancellation, completion, feedback, participant chat, simulated payment checkout and analytical reporting. Java and MySQL maintain the application workflow and persistent records, while Python provides recommendation scoring, feedback training and exploratory forecasting. The front end presents these capabilities through a common interface suitable for demonstration and further development in Eclipse.

Functional verification produced 59 passing automated checks across six suites, with additional browser checks for rendering and mobile layout. The results confirm behaviour for controlled local cases, including final-seat concurrency, automatic discovery of a newly offered ride, location-dependent pricing, protected chat access and simulated payment refunds. The recommendation system combines mandatory eligibility rules with a similarity baseline and an optional feedback-trained Random Forest contribution. The project does not report unmeasured predictive accuracy, guaranteed traffic reduction or actual financial settlement. Its current data includes synthetic demonstration records, and analytical savings remain estimates based on stated assumptions.

Future work should begin with consented, representative journey data and an independently held-out evaluation of ranking quality and forecast error. Useful extensions include route-overlap and detour-aware matching, recurring commute schedules, accessible multilingual interfaces and notifications when the browser is closed. A deployment study should measure response times, concurrent-user behaviour and the reliability of routing providers. Real payments would require a separate merchant integration with server-verified gateway callbacks, reconciliation and refund handling; the current simulator must not be presented as that integration. A production release would also require deployment-specific security, operational monitoring and data-retention decisions. These steps would extend the verified academic prototype into a more thoroughly evaluated service.

REFERENCES

[1] N. Agatz, A. Erera, M. Savelsbergh and X. Wang, "Optimization for dynamic ride-sharing: A review," *European Journal of Operational Research*, vol. 223, no. 2, pp. 295-303, 2012. https://doi.org/10.1016/j.ejor.2012.05.028

[2] P. Santi, G. Resta, M. Szell, S. Sobolevsky, S. H. Strogatz and C. Ratti, "Quantifying the benefits of vehicle pooling with shareability networks," *PNAS*, vol. 111, no. 37, pp. 13290-13294, 2014. https://doi.org/10.1073/pnas.1403657111

[3] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *PNAS*, vol. 114, no. 3, pp. 462-467, 2017. https://doi.org/10.1073/pnas.1611675114

[4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5-32, 2001. https://doi.org/10.1023/A:1010933404324

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. https://www.jmlr.org/papers/v12/pedregosa11a.html

[6] Project OSRM, "OSRM HTTP server: Route service," *OSRM API Documentation*. Accessed 22 October 2026. https://project-osrm.org/docs/v26.6.1/http

[7] Leaflet, "API reference," *Leaflet Documentation*. Accessed 22 October 2026. https://leafletjs.com/reference

[8] Oracle, "InnoDB Locking," *MySQL 8.0 Reference Manual*. Accessed 22 October 2026. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html

[9] Apache Software Foundation, "Apache Tomcat 10.1 Documentation Index." Accessed 22 October 2026. https://tomcat.apache.org/tomcat-10.1-doc/index.html